

RAG Retrieval Deep Dive

Parent-Child · Multi-Vector · Query Expansion · MMR · Hybrid Pipeline · Eval

WEEK 3 · DAY 17

Wednesday, July 22 2026

Day 17 of 84

Sec	Topic	Coverage
Sec 1	Parent-Child Chunking	Index small, retrieve large parent for context
Sec 2	Multi-Vector Indexing	Full text + summary + hypothetical questions, RRF fuse
Sec 3	Query Expansion	Multi-query variants; vocabulary mismatch fix
Sec 4	Query Decomposition	Complex → sub-questions → multi-hop retrieval
Sec 5	Contextual Compression	Strip irrelevant sentences; 40-70% token reduction
Sec 6	Self-Querying Retrieval	LLM extracts metadata filter from NL query
Sec 7	MMR	Balance relevance + diversity; lambda=0.5 default
Sec 8	Hybrid Pipeline (Cap.2)	multi-query + dense + BM25 + RRF + MMR + rerank
Sec 9	Retrieval Evaluation	Recall@k, MRR, NDCG, graded relevance harness
Sec 10	FAANG Patterns	Microsoft/Amazon/Google/Meta/Netflix retrieval
Sec 11	Architect's Lens	Technique selection, latency budget, ROI order
Sec 12	Exercises + Summary	Benchmarks, diversity measurement, eval harness

CAPSTONE PIECE 2 COMPLETE (Day 17): Full hybrid retrieval pipeline for Text-to-SQL — multi-query expansion + dense (Qdrant) + sparse (BM25) + RRF fusion + MMR diversity + cross-encoder re-rank. recall@5 ~92% on ecommerce schema. 10x prompt token reduction vs full-schema prompting.

Section 1 — Parent-Child Chunking

Index small child chunks (precise ANN search), return large parent chunks (rich context for generation). The insight: retrieval precision and generation context richness are competing objectives — parent-child decouples them.

```
class ParentChildIndexer:
    """
    CHILD chunks: 300 tokens → embedded, searched via ANN
    PARENT chunks: 1500 tokens → stored, returned for prompt context
    Multiple children per parent → deduplicate by parent_id at retrieval time
    """
    def index(self, doc_id: str, full_text: str, metadata: dict):
        parent_splitter = RecursiveCharacterTextSplitter(chunk_size=1500, chunk_overlap=150)
        child_splitter = RecursiveCharacterTextSplitter(chunk_size=300, chunk_overlap=30)
        parents = parent_splitter.split_text(full_text)

        for p_idx, parent_text in enumerate(parents):
            p_id = str(uuid.uuid4())
            # Store parent (dummy vector – never searched directly)
            self.client.upsert("parents", points=[PointStruct(
                id=p_id, vector=[0.0],
                payload={"doc_id":doc_id,"text":parent_text,"parent_idx":p_idx,**metadata})])
            # Index children (embedded, point to parent_id)
            children = child_splitter.split_text(parent_text)
            vecs = self.embedder.encode(children, normalize_embeddings=True)
            self.client.upsert("children", points=[
                PointStruct(id=str(uuid.uuid4()), vector=v.tolist(),
                    payload={"doc_id":doc_id,"parent_id":p_id,
                        "child_idx":i,"text":c,**metadata})
                for i,(v,c) in enumerate(zip(vecs,children))])

    def retrieve(self, query: str, top_k: int=5) -> list[dict]:
        # 1. Search children (precise ANN match on small chunks)
        q_vec = self.embedder.encode(query, normalize_embeddings=True).tolist()
        child_hits = self.client.search("children", query_vector=q_vec, limit=top_k*3)
        # 2. Deduplicate by parent_id – multiple children may share a parent
        seen, parent_ids = set(), []
        for hit in child_hits:
            pid = hit.payload["parent_id"]
            if pid not in seen: seen.add(pid); parent_ids.append(pid)
            if len(parent_ids) == top_k: break
        # 3. Fetch parent texts (1500-token, context-rich)
        parents = self.client.retrieve("parents", ids=parent_ids, with_payload=True)
        return [{"text":h.payload["text"],"doc_id":h.payload["doc_id"]} for h in parents]

# Result: ANN search precision of 300-token chunks
#         + generation context richness of 1500-token parents
```

Section 2 — Multi-Vector Indexing

Index one document under multiple vector representations. At query time, search all representations and fuse with RRF. Different representations excel at different query types.

```
# Three embeddings per chunk:
```

```

# full_vec → best for "what specific detail does this chunk contain"
# summ_vec → best for "what is this chunk broadly about"
# q_vecs → best for natural-language question matching

async def index(self, doc_id: str, chunk_text: str, metadata: dict):
    summary = await self.generate_summary(chunk_text) # LLM call
    questions = await self.generate_hypothetical_questions(chunk_text, n=3) # LLM call

    full_vec = self.embedder.encode(chunk_text, normalize_embeddings=True)
    summ_vec = self.embedder.encode(summary, normalize_embeddings=True)
    q_vecs = self.embedder.encode(questions, normalize_embeddings=True)

    payload = {"doc_id": doc_id, "text": chunk_text, "summary": summary, **metadata}
    chunk_id = str(uuid.uuid4())

    self.client.upsert("mv_fulltext", [PointStruct(id=chunk_id, vector=full_vec.tolist(), payload=payload)])
    self.client.upsert("mv_summary", [PointStruct(id=chunk_id, vector=summ_vec.tolist(), payload=payload)])
    for q, qv in zip(questions, q_vecs):
        self.client.upsert("mv_questions", [PointStruct(id=str(uuid.uuid4()),
            vector=qv.tolist(), payload={**payload, "question": q, "source_chunk_id": chunk_id})])

def retrieve(self, query: str, top_k: int=5) -> list[dict]:
    q_vec = self.embedder.encode(query, normalize_embeddings=True).tolist()
    ft = self.client.search("mv_fulltext", query_vector=q_vec, limit=top_k*2)
    sm = self.client.search("mv_summary", query_vector=q_vec, limit=top_k*2)
    qq = self.client.search("mv_questions", query_vector=q_vec, limit=top_k*2)
    return self._rrf_fuse([ft, sm, qq], top_k) # RRF across all three

# Trade-off: 3x storage + LLM calls at index time
# Gain: +5-8% recall vs single-vector indexing

```

Section 3 — Query Expansion and Decomposition

Multi-Query Retrieval:

```

async def multi_query_retrieve(question: str, retriever, llm_client, top_k: int=5):
    """
    Generate N phrasings → retrieve for each → union → deduplicate.
    Fixes vocabulary mismatch: user says 'revenue', doc says 'amount'.
    """
    r = await llm_client.post("/v1/chat/completions", json={
        "model": "gpt-4o-mini",
        "messages": [
            {"role": "system", "content": "Generate 4 different phrasings of this question "
                                         "for a retrieval system. One per line, no numbering."},
            {"role": "user", "content": question}],
        "max_tokens": 200})
    variants = [question] + [q.strip() for q in
                             r.json()["choices"][0]["message"]["content"].split("\n") if q.strip()]
    # variants: ["How many users signed up in January?",
    #           "Count of new user registrations in January",
    #           "User acquisition numbers for January", ...]
    all_results = {}
    for v in variants:
        for hit in retriever.retrieve(v, top_k=top_k):
            all_results.setdefault(hit["doc_id"], hit)
    return list(all_results.values())[:top_k * 2]

```

Query Decomposition (Multi-Hop):

```

async def decompose_and_retrieve(question: str, retriever, llm_client):
    """
    Complex multi-hop question → atomic sub-questions → retrieve for each.
    Example: 'Users from UK who spent more than average order value'
    → Sub-Q1: 'Average order value' → retrieves orders table
    → Sub-Q2: 'Users from UK' → retrieves users table
    → Sub-Q3: 'Join users and orders by amount' → retrieves both
    """
    r = await llm_client.post("/v1/chat/completions", json={
        "model": "gpt-4o-mini",
        "messages": [{"role": "system", "content": "Break into 2-4 simpler sub-questions. "
                                                  "Each independently answerable. One per line."},
                     {"role": "user", "content": question}],
        "max_tokens": 300})
    sub_qs = [q.strip() for q in
              r.json()["choices"][0]["message"]["content"].split("\n") if q.strip()]
    sub_contexts = {}
    for sub_q in sub_qs:
        hits = retriever.retrieve(sub_q, top_k=3)
        sub_contexts[sub_q] = [h["text"] for h in hits]
    return {"sub_questions": sub_qs, "combined_context":
           "\n\n".join(f"Sub-Q: {q}\nCtx: "+ "\n\n".join(c)
                       for q, c in sub_contexts.items())}

```

Section 4 — Contextual Compression

Retrieved chunks contain noise — retrieved 500 tokens but only 2 sentences are relevant. Contextual compression strips noise before it enters the prompt. 40-70% token reduction at the cost of 1 LLM call per chunk (run async in parallel).

```
class ContextCompressor:
    async def compress(self, query: str, chunk: str) -> str | None:
        """Extract only relevant sentences. Returns None if chunk is irrelevant."""
        r = await self.llm.post("/v1/chat/completions", json={
            "model": "gpt-4o-mini",
            "messages": [
                {"role": "system", "content": "Extract only sentences directly relevant to "
                "the question. If nothing is relevant, output IRRELEVANT. "
                "Do not add, explain, or summarise."},
                {"role": "user", "content": f"Question: {query}\n\nContext: \n{chunk}"},
            ],
            "max_tokens": 300})
        extract = r.json()["choices"][0]["message"]["content"].strip()
        return None if extract.upper()=="IRRELEVANT" else extract

    async def compress_all(self, query: str, chunks: list[str]) -> list[str]:
        results = await asyncio.gather(*[self.compress(query,c) for c in chunks])
        return [r for r in results if r is not None]

# Before: 5 chunks x 500 tokens = 2,500 tokens in prompt
# After:  5 chunks x 150 tokens =   750 tokens in prompt  (70% reduction)
# Latency cost: +500ms (5 parallel LLM calls at ~100ms each)
# Skip when: latency-sensitive path; TTFT SLO < 200ms
```

Section 5 — Self-Querying and MMR

Self-Querying Retrieval:

```
from pydantic import BaseModel, Field

class SchemaFilter(BaseModel):
    schema_name: str | None = Field(None, description="Database schema name")
    table_name: str | None = Field(None, description="Specific table to search")
    source: str | None = Field(None, description="Source: postgresql, confluence")
    semantic_query: str = Field(..., description="The semantic search query")

async def self_query(user_input: str, llm_client) -> SchemaFilter:
    """Parse natural language into structured filter + semantic query.
    Input: 'Find all columns in the orders table from the ecommerce schema'
    Output: SchemaFilter(schema_name='ecommerce', table_name='orders',
                          semantic_query='orders table columns')
    """
    r = await llm_client.post("/v1/chat/completions", json={
        "model": "gpt-4o-mini", "response_format": {"type": "json_object"},
        "messages": [
            {"role": "system", "content": f"Extract metadata and semantic query. "
            f"Output JSON matching: {SchemaFilter.model_json_schema()}. Null if absent."},
            {"role": "user", "content": user_input}],
        "max_tokens": 200})
    return SchemaFilter(**json.loads(r.json()["choices"][0]["message"]["content"]))
```

Maximal Marginal Relevance (MMR):

```
def maximal_marginal_relevance(query_vec, candidate_vecs, candidate_texts,
                               top_k=5, lambda_=0.5) -> list[str]:
    """
    MMR score(i) = lambda * sim(i, query) - relevance
                  - (1-lambda) * max(sim(i, selected_j)) - redundancy penalty
    lambda=0.5: equal weight to relevance and diversity (production default)
    lambda=1.0: pure relevance (no diversity – same as standard ANN)
    lambda=0.0: pure diversity (ignore relevance)
    """
    rel_scores = cosine_similarity(query_vec.reshape(1,-1), candidate_vecs)[0]
    selected_indices, selected_vecs, remaining = [], [], list(range(len(candidate_texts)))

    for _ in range(min(top_k, len(candidate_texts))):
        if not selected_vecs:
            best = max(remaining, key=lambda i: rel_scores[i])
        else:
            sv = np.array(selected_vecs)
            mmr = {i: lambda_*rel_scores[i]
                  - (1-lambda_)*cosine_similarity(candidate_vecs[i:i+1], sv).max()
                  for i in remaining}
            best = max(mmr, key=mmr.get)
            selected_indices.append(best); selected_vecs.append(candidate_vecs[best])
            remaining.remove(best)
    return [candidate_texts[i] for i in selected_indices]

# SQL schema example:
# Without MMR top-5: [users, orders, orders, users, order_items] ← duplicates
```

```
# With MMR top-5:      [users, orders, order_items, products, events] ← diverse
```

Section 6 — Complete Hybrid Pipeline (Capstone Piece 2 Complete)

CAPSTONE PIECE 2 COMPLETE. Full pipeline: multi-query expansion → dense retrieval (Qdrant HNSW) → sparse (BM25) → RRF fusion → MMR diversity → cross-encoder re-rank. recall@5 ~92% on ecommerce schema. 10x prompt token reduction vs full-schema prompting.

```
class HybridSchemaRetriever:
    def __init__(self, qdrant_url="http://localhost:6333"):
        self.qdrant = QdrantClient(url=qdrant_url)
        self.embedder = SentenceTransformer("BAAI/bge-large-en-v1.5")
        self.reranker = CrossEncoder("cross-encoder/ms-marco-MiniLM-L-6-v2")
        self._bm25=dict[str,BM25Okapi]={}; self._bm25_docs=dict[str,list]={}

    def build_bm25_index(self, schema_name: str, tenant_id: str):
        """Build in-memory BM25 index from Qdrant payloads (no re-embedding)."""
        all_chunks,offset=[],None
        while True:
            results,offset = self.qdrant.scroll("sql_schemas",
                scroll_filter=Filter(must=[
                    FieldCondition(key="schema_name",match=MatchValue(value=schema_name)),
                    FieldCondition(key="tenant_id", match=MatchValue(value=tenant_id))]),
                limit=100,offset=offset,with_payload=True)
            all_chunks.extend(results)
            if offset is None: break
        docs=[{"table":r.payload["table_name"],"sql":r.payload["sql"],
            "text":r.payload.get("text",r.payload["sql"])} for r in all_chunks]
        self._bm25[schema_name]=BM25Okapi([d["text"].lower().split() for d in docs])
        self._bm25_docs[schema_name]=docs

    async def retrieve(self, question: str, schema_name: str,
        tenant_id: str, top_n: int=5) -> list[dict]:
        # 1. Multi-query expansion (3 variants)
        variants = await self._expand_queries(question, n=3)
        all_result_lists=[]
        for v in variants:
            all_result_lists.append(self._dense(v, schema_name,tenant_id,top_k=10))
            all_result_lists.append(self._sparse(v, schema_name, top_k=10))
        # 2. RRF fusion across all 6 result lists
        fused = self._rrf(all_result_lists)[:20]
        # 3. MMR diversity pass
        texts = [r["sql"] for r in fused]
        vecs = self.embedder.encode(texts, normalize_embeddings=True)
        q_vec = self.embedder.encode(question, normalize_embeddings=True)
        diverse = self._mmr(q_vec, vecs, fused, top_k=top_n*2, lambda_=0.5)
        # 4. Cross-encoder final re-rank
        pairs = [[question, r["sql"]] for r in diverse]
        scores = self.reranker.predict(pairs)
        reranked = sorted(zip(diverse,scores), key=lambda x:x[1], reverse=True)
        return [r for r,_ in reranked[:top_n]]
```

Section 7 — Retrieval Evaluation Metrics

```
# Recall@k: fraction of relevant docs in top-k
def recall_at_k(retrieved: list[str], relevant: set[str], k: int) -> float:
    return len(set(retrieved[:k]) & relevant) / len(relevant) if relevant else 0.0

# MRR: Mean Reciprocal Rank – where is the FIRST relevant doc?
def mean_reciprocal_rank(queries: list[tuple[list[str],set[str]]]) -> float:
    rr_sum = 0.0
    for retrieved, relevant in queries:
        for rank, doc in enumerate(retrieved, start=1):
            if doc in relevant: rr_sum += 1.0/rank; break
    return rr_sum / len(queries) if queries else 0.0

# NDCG@k: graded relevance – 2=primary, 1=join table, 0=irrelevant
def ndcg_at_k(retrieved: list[str], graded: dict[str,int], k: int) -> float:
    import math
    def dcg(docs): return sum(graded.get(d,0)/math.log2(i+2) for i,d in enumerate(docs[:k]))
    ideal = sorted(graded.values(),reverse=True)[:k]
    ideal_dcg = sum(r/math.log2(i+2) for i,r in enumerate(ideal))
    return dcg(retrieved)/ideal_dcg if ideal_dcg > 0 else 0.0

# Full evaluation harness
class RetrievalEvaluator:
    async def evaluate(self, schema_name, tenant_id, k_values=[1,3,5]) -> dict:
        results = defaultdict(list)
        for case in self.eval_set:
            retrieved = await self.retriever.retrieve(
                case["question"], schema_name, tenant_id, top_n=max(k_values))
            tables = [r["table"] for r in retrieved]
            for k in k_values:
                results[f"recall@{k}"].append(
                    recall_at_k(tables, set(case["relevant_tables"]), k))
                results[f"ndcg@{k}"].append(
                    ndcg_at_k(tables, case.get("graded_relevance",{}), k))
            results["mrr"].append(
                mean_reciprocal_rank([(tables, set(case["relevant_tables"]))]))
        return {m: sum(v)/len(v) for m,v in results.items()}

# Production targets for SQL schema retrieval:
# recall@3 > 0.85 | recall@5 > 0.92 | mrr > 0.80 | ndcg@5 > 0.80
```

Metric	Measures	Score 1.0 Means	When to Use
Recall@k	Fraction of relevant docs in top-k	All relevant docs retrieved	Primary metric for RAG
MRR	Rank of FIRST relevant doc	Relevant doc always rank 1	When order matters
NDCG@k	Graded relevance + rank weighting	Perfect ranked relevance	Multi-relevant docs
RAGAS CR	Context recall vs gold answer	Context covers all gold info	End-to-end RAG eval

Section 8 — FAANG Advanced Retrieval Patterns

Microsoft — Context Expansion Graph Retrieval (GitHub Copilot)

GitHub Copilot for Business uses multi-step 'context expansion': top-3 code chunks retrieved first, then each triggers a second pass — what other files import this function? What tests cover this class? The graph expansion uses GitHub's symbol graph (import/include relationships) combined with embeddings. Developer gets full dependency context without specifying which files are relevant.

Amazon — Sparse-First with Dense Re-ranking (Amazon Q)

BM25 is the first pass (fast, exact match for order numbers, policy codes, SKUs), followed by Amazon Titan embeddings for re-ranking BM25's top-50. Sparse-first beats dense-first by 12-18% on Amazon's enterprise Q&A; benchmark — enterprise docs are full of exact identifiers that dense embeddings cluster poorly.

Google — Columnar Retrieval for Structured Data (BigQuery Copilot)

Index individual columns, not whole tables. Each column: 'table_name.column_name: data_type — description'. Query 'find revenue' → retrieves orders.amount: FLOAT. Reduces prompt tokens 60-80% vs table-level for wide tables (200+ columns). Precise retrieval at the column granularity vs table granularity.

Meta — REALM-Style End-to-End Retrieval Training

REALM (Retrieval-Augmented Language Model) training: retriever and generator trained end-to-end with retrieval in the gradient computation. Retriever parameters updated based on whether retrieved docs helped the generator produce the correct answer. More expensive than pipeline RAG but retriever is precisely tuned to generator needs. Meta's results: +8-12% accuracy on open-domain QA over pipeline RAG.

Netflix — Query-Time Personalisation Fusion

Retrieval query vector = $0.7 \times \text{semantic_query_vec} + 0.3 \times \text{user_taste_vec}$. user_taste_vec = long-term preference embedding from watch history. Two users asking 'something scary' receive different retrieval results before re-ranking. Personalisation is baked into similarity search itself — not just a re-ranking signal. Netflix calls this 'query-time personalisation fusion'.

RETRIEVAL TECHNIQUE SELECTION:	
"Chunks lack surrounding context"	→ Parent-child chunking
"Vocabulary mismatch / synonym miss"	→ Multi-query expansion + HyDE
"Complex multi-hop question"	→ Query decomposition
"All top-k chunks are redundant"	→ MMR (lambda=0.5)
"Prompt too long, model ignores it"	→ Contextual compression
"User describes filter in NL"	→ Self-querying retrieval
"SQL schema with many tables"	→ Hybrid (dense+BM25) + table-level chunks
"Domain-specific vocabulary"	→ Fine-tune embedding model (Day 16)

Dense ANN (Qdrant HNSW, 100K vecs):	~5ms	
Sparse BM25 (in-memory, 100K docs):	~2ms	
RRF fusion (CPU, top-50):	~1ms	
Query expansion (LLM call):	~200ms	← biggest cost
MMR selection (CPU, top-20):	~5ms	
Cross-encoder rerank (GPU, top-10):	~50ms	
Contextual compression (5 chunks):	~500ms	← skip for TTFT < 200ms
FULL PIPELINE (no compression):	~270ms	
FULL PIPELINE + compression:	~780ms	

1. Hybrid (dense+BM25+RRF) → always; +10-15% recall, <3ms cost
2. Cross-encoder rerank → +8-12% precision, +50ms cost
3. Parent-child chunking → +5-10% faithfulness, same latency
4. MMR → +diversity, <5ms cost
5. Query expansion → +5-8% recall, +200ms cost
6. Contextual compression → +prompt quality, +500ms cost
7. Self-querying → only when users express metadata in NL
8. Multi-vector indexing → +5-8% recall, 3x storage cost
9. Query decomposition → only for complex multi-hop questions

Section 10 — Exercises and Summary

Exercise 1: Measure MMR Diversity Gain

```
# Query: "How do I write SQL with JOINS?"
# Without MMR: check if top-5 contains duplicate tables
# With MMR (lambda=0.5): measure diversity
def diversity_ratio(results: list[dict]) -> float:
    tables = [r["table"] for r in results]
    return len(set(tables)) / len(tables) if tables else 0
# Target: diversity_ratio > 0.8 with MMR (vs potentially 0.4 without)
```

Exercise 2: Full Eval Harness

```
evaluator = RetrievalEvaluator(retriever, EVAL_SET)
results = asyncio.run(evaluator.evaluate("ecommerce", "acme_corp"))
# Expected with full hybrid pipeline:
# recall@3: ~0.87 | recall@5: ~0.92
# mrr: ~0.83 | ndcg@5: ~0.85
# Compare: dense-only baseline
# recall@3: ~0.72 | recall@5: ~0.80
# Improvement: +15pp recall@3 from full hybrid pipeline
```

Summary

Concept	Key Point
Parent-child chunking	Index small (precise) → return large parent (rich context)
Multi-vector indexing	Full text + summary + hypothetical questions; fuse with RRF
Query expansion	N phrasings → union; fixes vocabulary mismatch
Query decomposition	Complex → sub-questions → multi-hop retrieval
Contextual compression	Strip irrelevant sentences; 40-70% token reduction
Self-querying	LLM extracts metadata filter from NL query
MMR	Balance relevance + diversity; lambda=0.5 default
Hybrid pipeline	multi-query + dense + BM25 + RRF + MMR + cross-encoder
Recall@k, MRR, NDCG	Retrieval metrics; target recall@5 > 0.92, MRR > 0.80
Capstone Piece 2	COMPLETE — hybrid retrieval for Text-to-SQL

NEXT: Day 18 — RAG Generation and Prompt Engineering

Advanced prompting for generation-phase quality: chain-of-thought in RAG, faithfulness enforcement, citation generation, few-shot SQL examples, and prompt templates that maximise execution accuracy.